

OrderPulse — Step 4: Order Service & Asynchronous Messaging

Technical Instruction & Execution Blueprint for AI Developer Agents

OBJECTIVE

Build the Order Service and Worker Service. Interconnect gRPC stock reservation with asynchronous event streaming (Kafka) and background job processing (RabbitMQ).

Step 4 Tasks

- **1. Service Structure:** Setup cmd/order/ and cmd/worker/ with Uber Fx modules.
- **2. Messaging Producers:** Configure Kafka producer for event streams and RabbitMQ client for task queues.
- **3. Order Flow:** Implement gRPC call to Inventory Service, Postgres record creation, Kafka event emit (order-events), and RabbitMQ task enqueue (invoice_queue).
- **4. Worker Consumer:** Implement RabbitMQ consumer in Worker Service to process invoice background tasks.
- **5. Observability:** Tag all logs with unified trace_id and order_id using uber-go/zap.

Directory Layout

```
orderpulse/
├── cmd/
│   ├── order/
│   │   └── main.go
│   └── worker/
│       └── main.go
├── internal/
│   ├── order/
│   │   ├── config/
│   │   ├── grpc/
│   │   ├── kafka/
│   │   ├── rabbitmq/
│   │   ├── service/
│   │   └── module.go
│   └── worker/
│       ├── config/
│       ├── consumer/
│       └── module.go
```

Verification Commands

```
# Start Order Service & Worker Service
go run cmd/order/main.go
go run cmd/worker/main.go

# Trigger CreateOrder via gRPC
grpcurl -plaintext -d '{"user_id": "usr-1", "product_id": "prod-101", "quantity": 1}' localhost:5005

# Verify Kafka message stream
docker exec -it orderpulse-kafka kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic
```